

1.Explain what statelessness means in the context of REST and describe how this principle affects the way your food delivery API handles user session data between requests.

Ans=**Statelessness in REST**

Statelessness means that every request sent from the client to the server is **independent**. The server does **not** remember any information about previous requests. Each request must include all the information needed to process it, such as authentication details or request data.

How it affects a Food Delivery API

In a food delivery API, the server does not store user session data between requests.

For example:

- When a user logs in, they receive an authentication token (such as a JWT or API token).
- Every future request (view restaurants, add items to the cart, place an order, check order status) must include that token.
- The server verifies the token for each request instead of remembering the user's session.

1. Login Request

2. Get Restaurant List

3. Place Order

Each request includes the token because the server does not remember previous requests.

Advantages of Statelessness

- **Scalable:** Any server can handle any request because no session data is stored.
- **Simple:** The server does not need to manage user sessions.
- **Reliable:** If one server fails, another server can process the next request.
- **Easy to maintain:** Reduces server-side complexity.

2. Justify why ModelSerializer is the better choice for this use case, and describe at least two specific field-level validations you would add to protect the data integrity of menu items.

Ans=

Why ModelSerializer is the Better Choice

ModelSerializer is the best choice because it automatically creates serializer fields based on the Django model. This reduces code, saves development time, and makes the serializer easier to maintain.

Advantages of ModelSerializer:

- Automatically generates fields from the model.
- Includes built-in create() and update() methods.
- Requires less code than a regular Serializer.
- Easy to maintain when model fields change.
- Supports model validation automatically.

Field-Level Validations for Menu Items

1. Validate the Price

Ensure that the menu item's price is greater than zero.

```
def validate_price(self, value):
```

```
    if value <= 0:
```

```
        raise serializers.ValidationError("Price must be greater than 0.")
```

```
    return value
```

Purpose: Prevents invalid prices such as 0 or negative values.

2. Validate the Item Name

Ensure that the menu item name is not empty and has a minimum length.

Purpose: Prevents blank or very short names like "A" or " ".

3.Explain how refactoring this to use a `ModelViewSet` with a `DefaultRouter` reduces URL management overhead, and identify which HTTP methods are automatically handled by `ModelViewSet`.

Ans=

Using a **ModelViewSet** with a **DefaultRouter** in Django REST Framework makes your code simpler because you do not need to create separate URL patterns for each API operation.

Explanation

Normally, if you use `APIView` or generic views, you need to write each URL manually.

Example:

```
urlpatterns = [
    path('users/', UserList.as_view()),
    path('users/<int:pk>', UserDetail.as_view()),
]
```

How it reduces URL management overhead

- You write less code.
- No need to create separate URLs for Create, Read, Update, and Delete.
- URLs are generated automatically.
- Easier to maintain when the project grows.
- Reduces the chance of URL configuration mistakes.
- Makes the project cleaner and follows REST API standards.

HTTP methods automatically handled by `ModelViewSet`

4. Identify the most likely cause of this performance issue and justify which DRF pagination class you would choose to fix it. Compare the trade-offs between `PageNumberPagination` and `CursorPagination` for this scenario.

Ans=

Most Likely Cause of the Performance Issue

The most likely cause is that the API is returning **all menu items in a single response**. When there are thousands of records, the server takes longer to fetch the data, uses more memory, and sends a very large response to the client, making the API slow.

Why this happens:

- Large database queries take more time.
- Large JSON responses increase network usage.
- More memory is required on the server.
- The client takes longer to load and display the data.

Best DRF Pagination Class

For a food delivery API with a large number of menu items, **CursorPagination** is the better choice.

Why CursorPagination?

- Works efficiently with large datasets.
- Provides faster performance than page-number pagination on very large tables.
- Prevents duplicate or missing records when new menu items are added or deleted while users are browsing.
- Ideal for APIs that continuously receive new data.

Comparison: PageNumberPagination vs CursorPagination

Feature	PageNumberPagination	CursorPagination
URL Example	/menu/?page=2	/menu/?cursor=abc123
Easy to Understand	Yes	No
Performance on Large Data	Good	Excellent
Supports Direct Page Numbers	Yes	No
Handles Data Changes Well	May show duplicates or skip records	Yes

Feature	PageNumberPagination	CursorPagination
Best Use Case	Small or medium datasets	Large datasets and high-traffic APIs

Conclusion

The performance issue is most likely caused by returning **too many records in one response**. Using **CursorPagination** is the best solution for a large food delivery API because it is faster, scales better with large datasets, and provides more consistent results than **PageNumberPagination** when data changes frequently.

5. Identify the specific gap in the current permission setup and explain how you would implement object-level permission to ensure each customer can only access their own orders

Ans=

Gap in the Current Permission Setup

The main gap is that the API only checks whether the user is **authenticated** (for example, using `IsAuthenticated`). It does **not** check whether the logged-in user is the **owner** of a specific order.

This means a customer could access another customer's order by changing the order ID in the URL, such as:

If order **5** belongs to another customer, the API could still return it unless object-level permissions are enforced.

Implementing Object-Level Permission

Create a custom permission class that allows users to access only their own orders.

```
from rest_framework.permissions import BasePermission
```

```
class IsOrderOwner(BasePermission):
```

```
    def has_object_permission(self, request, view, obj):
```

```
        return obj.customer == request.user
```

Why This Improves Security

- Users must be logged in (IsAuthenticated).
- Users can only view, update, or delete **their own** orders.
- Prevents unauthorized access to other customers' data.
- Protects customer privacy and maintains data integrity.

6. Describe how you would integrate the Google Maps Geocoding API into a DRF view to convert an address string into coordinates and use those coordinates to return nearby restaurant results.

Ans=

Integrating Google Maps Geocoding API in a DRF View

The **Google Maps Geocoding API** converts a user's address (for example, "Ahmedabad, Gujarat") into **latitude** and **longitude** coordinates. These coordinates can then be used to find nearby restaurants stored in the database.

Steps

1. The client sends an address to the API.
2. The DRF view calls the Google Maps Geocoding API.
3. Google returns the latitude and longitude.
4. The API searches for restaurants near those coordinates.
5. The nearby restaurant list is returned to the user.

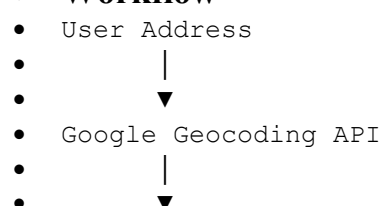
Finding Nearby Restaurants

After getting the coordinates, calculate the distance between the user's location and each restaurant.

You can:

- Store each restaurant's **latitude** and **longitude** in the database.
- Use **GeoDjango**, **PostGIS**, or a distance formula (such as the Haversine formula) to find restaurants within a specific radius (for example, **5 km**).

• Workflow



- Latitude & Longitude
- |
- ▼
- Search Restaurants Nearby
- |
- ▼
- Return Restaurant List

Advantages

- Converts any valid address into accurate coordinates.
- Helps users find restaurants close to their location.
- Improves search accuracy compared to searching by address text.
- Supports location-based features such as nearby restaurants, delivery zones, and estimated delivery distance.